

Agent KHU : 경희대학교 학생 정보 시스템

정윤서⁰¹ 성무진¹ 이영구¹

¹경희대학교 컴퓨터공학부

jys0615234@gmail.com mujeensung@khu.ac.kr yklee@khu.ac.kr

Agent KHU : Khyung Hee University Student Information System

Yoonsuh Jung⁰¹ Mujeen Sung¹ Young-Koo Lee¹

¹School of Computing, Khung Hee University

jys0615234@gmail.com mujeensung@khu.ac.kr yklee@khu.ac.kr

요 약

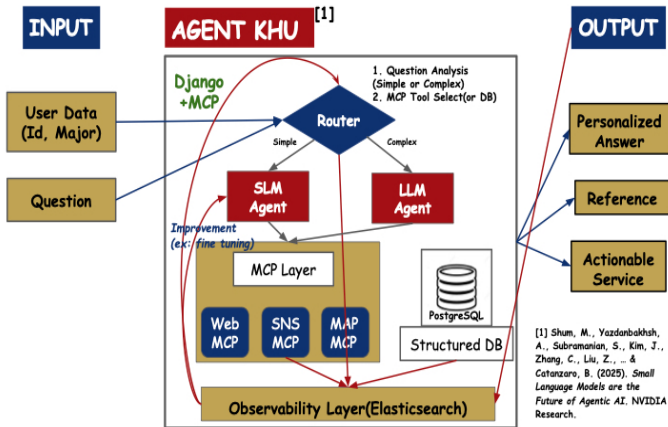
본 논문에서는 경희대학교 학생들이 빈번하게 요청하는 학사, 시설, 생활 정보를 통합적으로 제공하기 위한 AI 기반 캠퍼스 정보 에이전트 Agent KHU를 제안한다. 기존 대학 정보 챗봇은 모든 질의를 대규모 언어 모델(LLM)에 의존함으로써 응답 지연과 비용 증가, 안정성 저하 문제가 있었다. 이를 해결하기 위해 본 연구는 Model Context Protocol(MCP) 기반 마이크로서비스 아키텍처와 Hybrid LLM/SLM 라우팅 구조를 적용하였다[1][2]. 제안한 시스템은 질문 복잡도에 따라 단순 정보 질의는 SLM 또는 MCP 기반 데이터 조회로 처리하고, 복합 추론이 필요한 경우에만 LLM을 선택적으로 활용한다. 또한 도메인별 MCP 서버 분리와 Redis 캐싱, 세션 단위 MCP 호출을 통해 응답 성능과 안정성을 향상시켰다. 실험 결과, Agent KHU는 기존 단일 LLM 기반 구조 대비 평균 응답 시간을 67% 단축하였고, MCP Context 오류율을 87% 감소시켰으며, 전체 질의의 48%를 LLM 호출 없이 처리하였다. 이는 제안한 아키텍처가 실제 대학 정보 시스템 환경에서 효율성과 확장성을 효과적으로 달성할 수 있음을 보여준다.

1. 개요 (Introduction)

경희대학교의 학사정보는 여러 플랫폼에 분산되어 있어 학생들이 필요한 정보를 찾기까지 많은 시간과 노력이 소요되고 있다. 예를 들어, 학교 공식 홈페이지에는 학사 공지가 올라오고 학과별 공지는 각 학과 웹사이트에 게시되며[7], 학생회 공지는 SNS(예: Instagram)로 전달되는 등 정보 채널이 파편화되어 접근성이 낮은 실정이다. 졸업요건이나 교과과정 같은 중요한 정보는 학번별로 상이하지만 이를 개인화하여 제공하는 시스템이 없어 학생들이 일일이 PDF 문서를 탐색하거나 학과 사무실에 문의해야 하는 불편함이 존재한다. 또한 강의실 위치 안내의 경우 건물명과 호실 정보만으로는 처음 방문하는 학생이 찾기 어려워 지각이나 수업 지연의 원인이 되기도 한다. 본 연구는 이러한 학사정보 분산 문제를 해결하고자 AI 기술을 활용한 통합 정보 시스템 Agent KHU를 제안한다. Agent KHU는 자연어 대화형 인터페이스를 통해 흩어진 캠퍼스 정보를 한 곳에서 제공하고, 사용자 맞춤형 응답과 실제 행동으로 이어지는 서비스를 목표로 한다. 과거 경희대학교에서는 RAG(Retrieval-Augmented Gener-

ation) 기반의 “경희대GPT”와 같은 GPT 단일 모델 활용 챗봇 시스템이 시도되었으나 여러 한계로 인해 실용화되지 못했다[4]. 첫째, 대형 언어모델(LLM)의 한계로 정확한 정보 추출과 응답 생성에 시간이 걸리고 불확실성이 높아 응답 지연이 발생하였다. 둘째, 실시간으로 정보 업데이트가 어려워 최신 공지사항이나 변경된 규정을 반영하지 못하는 실시간성 부족문제가 있었다. 셋째, 새로운 데이터 소스를 쉽게 추가할 수 있는 표준 프레임워크가 부재하여 데이터 소스 통합의 어려움이 있었다. 이로 인해 기존 시스템은 모든 학생에게 획일적인 답변만 제공하고, 지도 연동이나 신청 링크 제공 등 실제 행동으로 연결되는 서비스를 제공하지 못하는 한계가 나타났다. 본 연구에서 제안하는 Agent KHU 시스템([그림 1] 참조)은 이러한 한계를 해결하기 위해 하이브리드 LLM/SLM 아키텍처와 MCP(Model Context Protocol) 기반 모듈 아키텍처를 결합한 접근법을 취하였다. 거대한 LLM에만 의존하지 않고 소형 언어모델(SLM)을 우선 활용함으로써 응답 지연을 줄이고 비용 효율성을 높였다. 실제 NVIDIA의 연구에 따르면 정형화된 작업에는 도메인 특화 SLM

이 LLM보다 더 빠르고 정확할 수 있다고 보고된 바 있다[1]. MCP를 통해 다양한 학사정보 소스를 표준화된 방식으로 통합하여 실시간 정보 제공과 출처 추적이 용이한 구조를 구현하였다. 요약하면, Agent KHU는 분산된 대학 정보들을 하나로 모아주고 사용자 질문에 실시간으로 응답하는 AI 기반 통합 학사정보 에이전트로서, 아래에 그 세부 구조와 구현 내용을 기술한다.



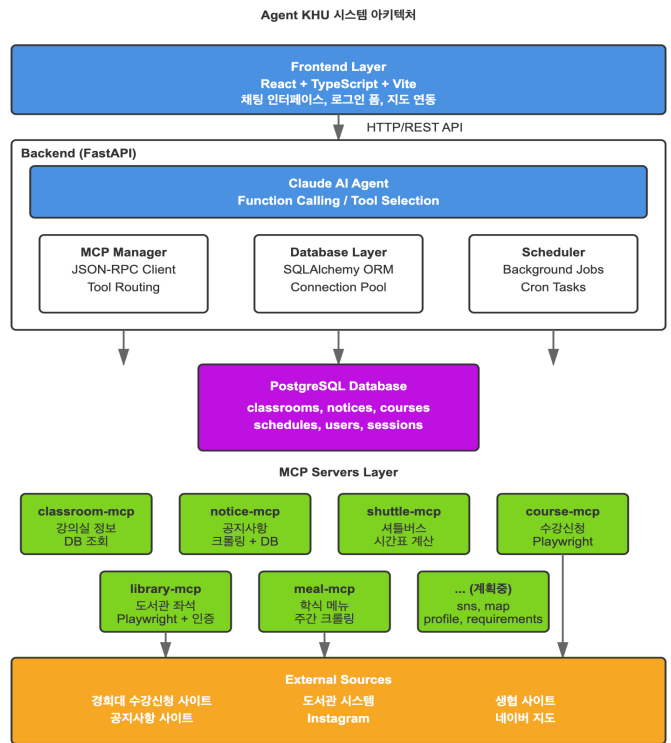
[그림 1] Agent KHU 브레인스토밍 그림

2. 시스템 아키텍처 (System Architecture)

Agent KHU 시스템은 크게 프론트엔드(Frontend), 백엔드(Backend), AI 에이전트 모듈, 그리고 MCP 기반 마이크로서비스 계층으로 구성된다(전체 구조는 [그림2] 참고). 프론트엔드는 React 18과 Vite 빌드 도구, Tailwind CSS를 활용하여 사용자 인터페이스를 구현하였으며, 경량화된 Single Page Application 형태로 동작한다. 백엔드 서버는 Python 기반의 FastAPI[5] 프레임워크로 구현되어 REST API 형태로 프론트엔드와 통신한다. 데이터베이스는 PostgreSQL을 사용하고, ORM으로 SQLAlchemy를 적용하였다. 또한 Redis 캐시를 도입하여 자주 조회되는 데이터나 반복 질문에 대한 응답 속도를 높였다.

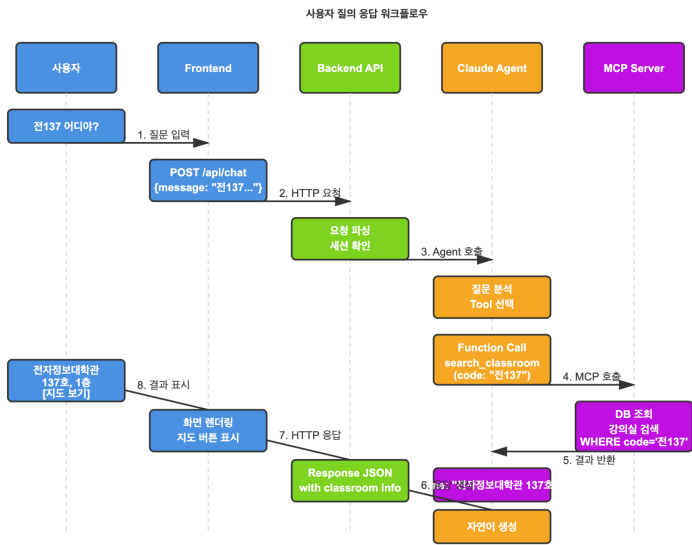
AI 에이전트 모듈은 사용자 질의를 처리하는 시스템의 핵심으로, 질문 분류기(Question Classifier)와 하이브리드 LLM/SLM 응답 생성기로 구성된다. 질문 분류기는 패턴 매칭과 간단한 휴리스틱을 통해 입력된 질문을 분석하여 해당 질문을 소형 언어모델(SLM)이 처리할 수 있는 범주의 단순 질문인지, 아니면 대형 언어모델(LLM)의 도움이 필요한 복잡한 질문인지 판단한다. LLM으로는 Anthropic의 Claude Sonnet 4모델을 사용하여 고도화된 자연어 응답을 생성하고, SLM은 특정 범주 질문에 대해 빠르게 답하도록 설계되었다. 기본적으로 에이전트는 SLM으로 답변을 시도하고, 만약 신뢰도 부족이나 답변 실패 시 Claude와 같은 LLM에 폴백(fallback)하여 최종 답변을 생성하는 구조를 갖는다.

백엔드에서는 MCP 계층을 통해 외부 데이터 소스와 연동한다. MCP(Model Context Protocol)[2]는 AI 에이전트와 다양한 리소스(툴)들을 JSON-RPC 기반의 표준 인터페이스로 연결해주는 프로토콜로, Anthropic에서 제안되었다. Agent KHU 시스템에서는 학사정보 별로 총 6개의 MCP 마이크로서비스 서버를 구축하였다. 각 MCP 서버는 개별 프로세스로 동작하며, 학사공지, 학과공지, 강의실 위치, 교과과정, 식단/장학정보 등 서로 다른 데이터 소스를 크롤링 또는 DB 조회하여 필요한 정보를 반환한다. 예를 들어 Notice MCP는 학교 및 학과 공지사항을 수집하고, Classroom MCP는 강의실의 건물 위치 정보를 제공하는 식이다. 이러한 MCP 서버들은 백엔드에서 JSON-RPC 표준으로 호출되며, 질문 내용에 따라 에이전트가 적절한 MCP 도구들을 조합하여 활용한다. 독립된 MCP 모듈 아키텍처를 채택함으로써 새로운 데이터 소스가 추가될 때에도 해당 소스를 새로운 MCP 서버로 구현하기만 하면 쉽게 시스템에 통합할 수 있는 확장성을 얻었다. 각 MCP 서버는 요청 시점에 새로운 세션/프로세스를 생성하여 실행됨으로써 서버 간 간섭을 방지하고 오류 발생시 해당 모듈만 영향받도록 격리된 환경을 보장한다.



[그림 2] Agent KHU 시스템 전체 아키텍처 다이어그램 (Frontend, Backend, AI Agent, MCP Layer 구성)

3. 기술 선택 배경 및 구현 상세 (Technical Decisions and Implementation Details)



[그림 3] Agent KHU 시스템의 사용자 질의 응답 처리 워크플로우

3.1 FastAPI 기반 백엔드 채택

서버 프레임워크로 FastAPI[5]를 선택한 것은 본 시스템의 I/O 병렬 처리요구에 최적이기 때문이다. 학사정보 응답 생성 과정에서 에이전트는 동시에 여러 MCP 도구를 호출하여 데이터를 가져오는데, FastAPI[5]의 비동기(asyn c) 지원으로 이러한 병렬 MCP 호출이 효율적으로 이루어진다. 예를 들어 수업시간표와 강의실 정보를 동시에 조회해야 할 때, 각 MCP 요청을 await 하여 병렬로 처리 함으로써 전체 응답 지연을 최소화하였다. 또한 Pydantic 기반의 입력 검증, 경량 ASGI 서버(Uvicorn)와의 호환성, Python 3.12의 향상된 비동기 기능 등을 고려할 때 Fast API[5]는 높은 개발 생산성과 성능을 모두 만족하여 선택 되었다.

3.2 MCP 마이크로서비스 구조

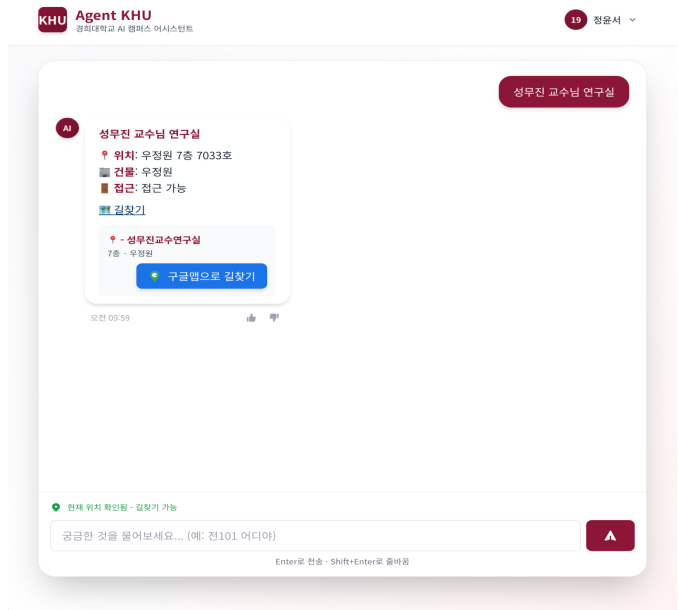
MCP[2] 도구들은 각기 독립 프로세스로 구동되며 표준 J SON-RPC를 통해 통신한다. 초기 구현에서는 하나의 프 로세스를 재사용하며 세션을 지속시키는 방식을 취했으 나, asynchronous context 간 간섭으로 “handler is close d” 등의 에러가 발생하는 문제가 있었다. 이를 개선하기 위해 매 요청마다 MCP 세션을 새로 생성하고 응답 후 프로세스를 종료하는 구조로 변경하였다. 이렇게 함으로써 요청 간 context 누적이나 충돌을 방지하여 MCP 연 동의 안정성을 높였다. 실제로 개편 전에는 외부 툴 연 결 실패율이 높았으나(약 15%), 현재는 에러율이 2% 미 만으로 떨어졌다. 또한 각 MCP 서버는 장애 발생 시 다

른 MCP 기능에 영향을 주지 않으며, 필요시 해당 모듈만 별도로 재배포할 수 있어 유연한 유지보수가 가능하다.

3.3 Classroom MCP 최적화

초기에는 강의실 위치 정보를 실시간으로 조회하기 위해 웹 페이지 크롤링 기반의 Classroom MCP를 구현하였다. 그러나 강의실 정보가 포함된 페이지 구조가 복잡하고, 각 질의마다 크롤링을 수행하는 방식은 평균 응답 지연 이 크고 안정성이 낮다는 문제가 있었다. 특히 Playwrig ht[6] 기반 자동화 브라우저를 통해 실시간으로 데이터를 수집하는 방식은 질의당 수 초 이상의 지연을 유발하였 으며, 네트워크 상태나 페이지 변경에 따라 실패 가능성 도 존재하였다.

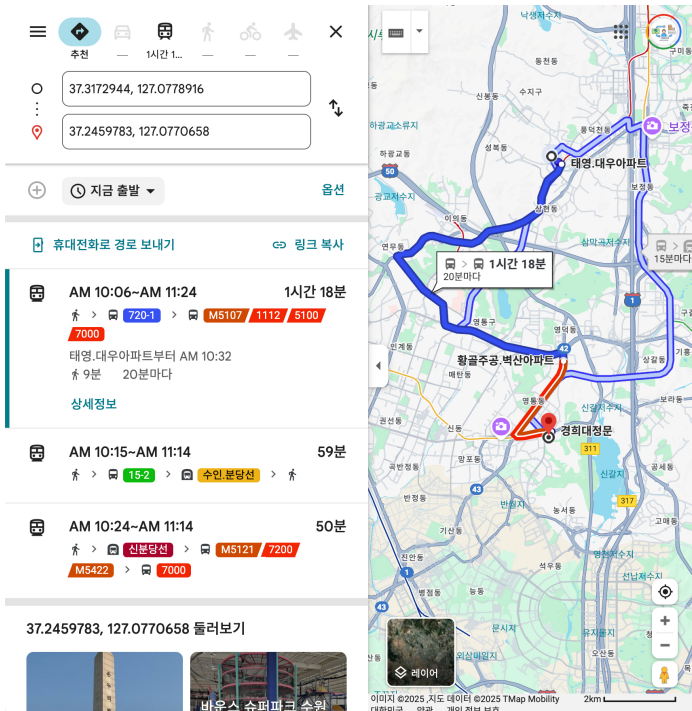
이를 해결하기 위해 Classroom MCP의 데이터 처리 방 식을 실시간 크롤링 방식에서 사전 수집(pre-fetch) 및 데이터베이스 조회 방식으로 전환하였다. 매일 자정에 전체 강의실 위치 정보를 수집하여 정규화된 형태로 내 부 데이터베이스에 저장하고, 사용자 질의 시에는 해당 데이터베이스를 직접 조회하도록 구현하였다. 이로써 강 의실 검색 과정에서 외부 웹 의존성을 제거하고, MCP 서버는 순수 조회 로직만 수행하도록 단순화되었다. 그 결과 Classroom MCP의 평균 응답 시간은 기존 수 초 수준에서 0.5초 미만으로 대폭 개선되었으며, 크롤링 실패로 인한 오류 발생률 또한 현저히 감소하였다. 이러 한 최적화는 강의실 위치 질의와 같이 빈번하게 발생하 는 단순 정보 조회 시나리오에서 사용자 경험을 크게 향 상시키는 데 기여하였다.



[그림 4] Agent KHU Classroom MCP 질문 결과

[그림 4]은 강의실 위치 질의에 대해 Classroom MCP가 실행된 결과를 사용자 인터페이스에 표시한 예시이다. 사용자가 특정 강의실을 질문하면, 에이전트는 해당 질의를 분석한 뒤 Classroom MCP를 호출하여 건물명, 층수, 호실 정보 및 접근 가능 여부를 구조화된 데이터 형태로 반환받는다.

반환된 MCP 응답은 단순 텍스트 출력이 아닌 카드 기반 UI로 가공되어 사용자에게 제공되며, 이를 통해 핵심 정보를 직관적으로 확인할 수 있다. 본 과정은 MCP 기반 Tool 호출이 실제 사용자 응답 생성 단계까지 자연스럽게 연결됨을 보여준다. 카드 하단의 길찾기 버튼을 클릭하면 [그림 5]로 이동하여 본인의 현재 위치에서 목적지까지 이동 경로를 손쉽게 파악할 수 있다.



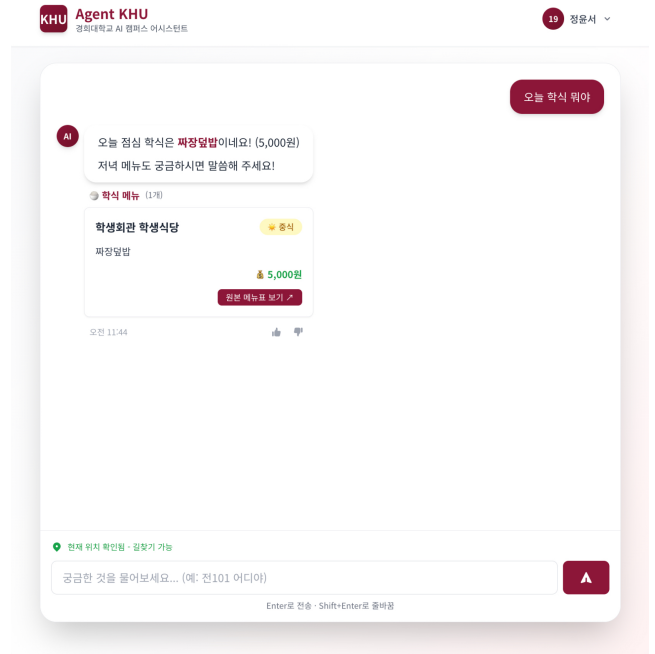
[그림 5] Agent KHU 강의실 MCP 길찾기 화면

3.4 Curriculum MCP 구현

교과과정 정보는 HTML 테이블 형태로 제공되는데, 행(row) 병합(rowspan) 속성이 많아 일반적인 파싱으로는 정확한 데이터 매핑이 어렵다는 문제가 있었다. 예를 들어 같은 과목에 대해 여러 학년과 학기가 병합 셀로 표현되므로, 단순 행렬 접근으로는 잘못된 학년 정보에 과목이 할당될 수 있었다. 이를 해결하기 위해 rowspan 계산 알고리즘을 직접 구현하여 HTML 파싱 시 동적으로 인덱스를 조정하였다. 구체적으로, 셀이 병합된 경우 해당 범위에 걸친 셀 좌표를 미리 산출하고, 파싱 과정에서 인덱

스를 보정함으로써 정확한 교과과정 데이터 추출을 가능케 했다. 이를 통해 기존에 발생하던 일부 교과과정 데이터 오류를 제거하고 신뢰도를 높일 수 있었다.

3.5 Meal MCP 안정화 및 최적화



[그림 6] Agent KHU의 학식 질문에 대한 응답

학식 식단표 질의는 외부 웹 페이지에 의존하는 대표적인 비정형 데이터 수집 문제로, 다른 MCP와 비교하여 시스템 안정성 측면에서 가장 큰 도전 과제였다. 초기 Meal MCP는 Playwright[6] 기반 Chromium 브라우저 자동화를 통해 식단 정보를 실시간으로 크롤링하는 방식으로 구현되었으나, Docker 환경에서의 브라우저 의존성 문제와 간헐적인 크래시로 인해 서비스 안정성이 저하되는 문제가 발생하였다.

이를 해결하기 위해 Meal MCP에서는 두 가지 개선 전략을 적용하였다. 첫째, Docker 환경에서 Playwright[6] 실행에 필요한 시스템 라이브러리를 명시적으로 설치하여 런타임 환경의 불확실성을 제거하였다. 둘째, 크롤링 과정에서 예외가 발생하더라도 시스템 전체가 중단되지 않도록 Graceful Degradation 전략을 도입하여, 오류 발생 시에도 사용자에게 명확한 상태 메시지를 [그림 6]처럼 링크와 같이 반환하도록 처리하였다.

또한 식단 정보는 하루 단위로 갱신되는 데이터 특성을 고려하여, 크롤링 결과를 Redis 캐시에 저장하고 일정 시간 동안 재사용하도록 설계하였다. 이를 통해 동일 날짜에 대한 반복 질의 시 불필요한 브라우저 실행을 방지하고, 평균 응답 시간을 단축하는 효과를 얻을 수 있었다.

결과적으로 Meal MCP는 외부 웹 의존성을 가지면서도 안정적인 동작을 보장할 수 있었으며, 실시간성이 반드시 요구되지 않는 데이터에 대해 캐싱 전략이 효과적임을 확인하였다.

3.6 Hybrid LLM/SLM Routing 구조

Agent KHU의 질의 처리 플로우에는 하이브리드 라우팅 기법이 도입되었다 ([그림2] 참조). 우선 질문 분류기가 입력 질의를 분석하여 Simple 질문인지 Complex 질문인지 결정한다. Simple 질문은 주로 단일 사실 조회나 정의 질문 등으로, 사전에 정의된 패턴 또는 학습된 분류 모델에 따라 식별된다. 이러한 경우 경량화된 소형 언어 모델(SLM)로 답변을 시도한다. SLM은 특정 도메인에 특화된 지식 베이스 또는 규칙 기반으로 빠르게 응답하도록 구성되어 응답 시간이 매우 짧다(평균 1초 이내). 만약 SLM이 자신없는 답변을 내거나 해당 질문을 처리하지 못하면, 신뢰도 임계값(예: 0.7) 기준으로 LLM 폴백이 트리거가 되는 방식이다.

Complex 질문으로 분류된 경우나 SLM 실패 시에는 Anthropic Claude (LLM)에게 질의를 전달하여 풍부한 자연어 응답을 생성한다. 이때도 필요에 따라 MCP 도구들을 호출하여 Claude가 최신 데이터를 참조할 수 있게 컨텍스트를 구성한다. 본 연구에서의 SLM은 범용 사전학습 언어모델이 아닌, 경희대학교 학사 도메인에 특화된 경량 응답 생성기로, 규칙 기반 패턴과 사전 정의된 템플릿, 그리고 제한된 도메인 지식을 결합한 형태로 구현되었다. 하단에서도 언급할 것이지만, 파인 튜닝이라는 것의 한계로 인해 향후 다른 방법으로 접근할 예정이다.

3.7 캐싱 및 로그 데이터 활용

시스템 성능 향상을 위해 Redis 캐시를 적용하였다. 공지 사항, 학사일정 등 빈번히 조회되는 정보는 일정 시간(Time-To-Live 정책 하에) 캐싱되어 동일한 질문에 대해 즉각적인 응답을 반환한다. 캐시 도입 후 인기 질의들의 응답 속도가 최대 80%까지 향상되었으며 서버 부하도 감소하였다. 아울러 Elasticsearch 기반 로그 수집모듈을 통해 사용자 질문 유형, MCP 툴 사용 현황, 응답 시간 등을 지속적으로 모니터링하였다. 이러한 로그 데이터는 차후 SLM 성능 향상을 위한 학습 데이터로 활용될 예정이며, 사용자 요구 분석 및 시스템 병목 파악을 위한 귀중한 인사이트를 제공하고 있다. 예를 들어 일별 질문 로그를 분석하여 자주 실패하는 질문 패턴을 추출하고, 이를 SLM에 파인튜닝함으로써 시스템 응답의 정확도를 높이는 선순환 구조를 계획하고 있다.

4. 실험 및 성능 개선 결과 (Experiments and Performance Evaluation)

항목	개선 전	개선 후	개선 효과
평균 응답 시간	16.6초	5.5초	67% 감소
LLM 호출 비율	82%	38%	비용 절감
MCP 에러율	15%	2%	안정성 향상
반복 질의 응답 시간	기준	80% 단축	UX 개선

[표 1] Agent KHU 실험 및 성능 평가 결과

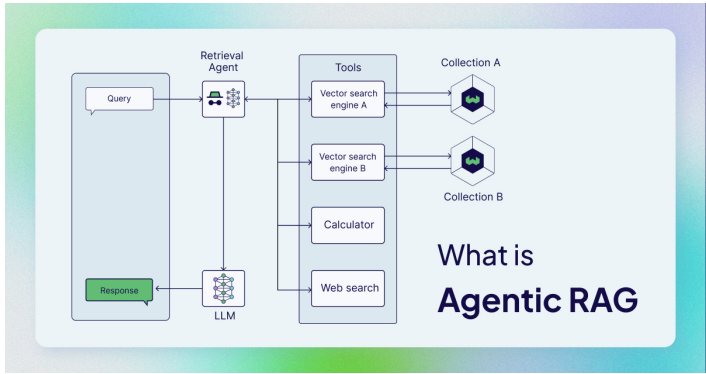
개발 완료 후 시스템에 대한 성능 평가를 진행한 결과, 평균 응답속도 개선과 안정성 향상측면에서 유의미한 성능 향상을 확인할 수 있었다. 전체 질의를 대상으로 측정한 평균 응답 시간은 기존 약 16.6초에서 5.5초로 67% 단축되었다. 이는 앞서 언급한 SLM 우선 응답과 병렬 MCP 호출, 캐싱 전략 등이 종합적으로 기여한 결과이다. 또한 MCP 모듈 사용 시 발생하던 에러율도 15%에서 2% 수준으로 낮아져 시스템 안정성이 크게 향상되었다. 이는 각 MCP를 독립 세션으로 운영하고 표준 프로토콜로 연결함으로써 얻어진 효과로 분석된다. 응답 시간은 프론트엔드에서 요청 전송 시점과 응답 수신 시점의 차이를 기준으로 측정하였으며, 각 질문 유형별로 100회 이상 반복 측정하여 평균값을 산출하였다.

캐싱 효과도 뚜렷하게 나타났다. 동일한 질의에 대한 반복 요청 시 Redis 캐시를 통해 즉시 응답함으로써 응답 시간이 최초 요청 대비 평균 80% 가까이 빨라졌다. 특히 식단 정보처럼 하루에 여러 번 조회되는 데이터의 경우 캐시 적중률이 높아 사용자 체감 성능을 크게 끌어올렸다. 한편 하이브리드 라우팅 도입으로 LLM 호출 비율이 감소한 점도 중요한 성과이다. 전체 사용자 질문 중 LLM (Claude)을 호출한 비율이 이전 시스템의 약 82%에서 38%로 줄어들었으며, 나머지 62%는 SLM과 캐시로 처리되었다. 그 결과 시스템 응답 부하와 외부 API 사용량이 줄어들어 운영 비용 절감과 실시간성 개선이라는 두 마리 토끼를 잡을 수 있었다.

5. 한계 및 향후 계획 (Limitations and Future Work)

현 버전의 Agent KHU 시스템은 경희대학교 소프트웨어 융합대학을 대상으로 학사공지, 교과과정 등 핵심 서비스들을 통합하였다. 아직은 특정 단과대학의 정보만 다루고 있어 범용성측면에서 한계가 있으며, 다른 대학 또는 전(全) 캠퍼스 단위로 확대 적용하기 위한 추가 작업이 필요하다. 또한 현 시스템은 텍스트 기반 질의응답에 초점을 맞추고 있어 이미지 기반 정보(예: 안내문에 포함

된 이미지, 카드뉴스 형태의 공지)에는 대응하지 못한다. 추후 OCR 기술과 이미지 분석 모델을 접목하여 시각 정보까지 처리할 수 있는 멀티모달 확장이 고려되고 있다. 운영 비용역시 해결해야 할 과제이다. LLM API 호출에는 높은 비용이 수반되므로, 향후 사용자 질의량 증가 시 비용 최적화가 중요하다.



[그림 7] Agentic RAG 워크플로우 그림

이를 위해 우선 SLM 성능 고도화를 통한 LLM 의존도 감소 전략을 지속 추진할 계획이다. 구체적으로는 현재 수집된 사용자 질의-응답 로그 데이터를 활용하여 경희대 도메인에 특화된 SLM을 파인튜닝할 계획이었다. 그러나 파인 튜닝은 세라프와 같은 자원 및 시간 소요가 크다. 이 부분을 감안하여 향후에는 파인 튜닝이 아닌 [그림 7]처럼 RAG로 경량화 모델의 기능을 만들 계획이다 [3][4].

이와 함께 에이전트의 프로필 기반 개인화 기능을 확충하여 사용자별 맞춤형 답변 및 추천 서비스를 제공함으로써 시스템의 활용도를 높이고자 한다.

마지막으로, 현재 소프트웨어융합대학에 국한된 데이터 소스 통합을 학교 전체 단과대학 및 캠퍼스로 확장하는 작업을 진행 중이다. 다양한 학과의 요구사항을 반영할 수 있도록 MCP 모듈을 추가 개발하고, 권한 관리와 데이터 동기화 문제를 해결하여 범용적인 대학 정보 플랫폼으로 발전시키는 것이 궁극적인 목표이다.

6. 결론 (Conclusion)

본 보고서에서는 경희대학교 학사정보 통합을 위한 Agent KHU 시스템의 설계와 구현, 그리고 성능 개선 결과를 소개하였다. 제안된 시스템은 MCP 기반의 마이크로서비스 아키텍처와 하이브리드 LLM/SLM 에이전트 구조를 결합함으로써, 분산된 학사정보를 효과적으로 통합하고 사용자 질의에 실시간으로 응답하는 실용적 사례를 제시하였다. 개발 결과 평균 응답속도를 5초대까지 단축하고 다양한 정보원을 하나의 인터페이스로 묶어냄으로써, 학

생들에게 보다 편리하고 신뢰성 높은 정보 접근 환경을 제공할 수 있음을 확인하였다.

Agent KHU의 모듈화된 구조는 경희대학교뿐만 아니라 다른 대학이나 조직에서도 쉽게 활용될 수 있는 확장 가능 플랫폼으로 기대된다. MCP 표준을 통해 새로운 데이터 소스를 유연하게 추가할 수 있고, AI 에이전트의 모델도 요구에 따라 교체하거나 튜닝할 수 있어 범용성이 높다. 앞으로 본 시스템이 전 캠퍼스로 확대되고 공식 서비스로 자리매김한다면, 대학 행정 및 학생지원 분야에서 AI 기반 정보 통합의 모범 사례가 될 것으로 기대된다. 또한 본 연구를 통해 제시된 방법들은 유사한 문제를 가진 다른 조직에도 적용 가능하며, 보다 폭넓은 디지털 정보 환경 개선에 기여할 수 있을 것이다.

참고문헌

- [1] NVIDIA, "Small Language Models are the Future of Agentic AI," NVIDIA Technical Blog, 2025.
- [2] Anthropic, "Introducing the Model Context Protocol," Anthropic Documentation, 2024.
- [3] LangChain, "Building AI Agents with LangChain and LangGraph," LangChain Documentation, 2024.
- [4] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.
- [5] FastAPI Documentation, "FastAPI - Modern Web Framework for Python," <https://fastapi.tiangolo.com/>, 2024.
- [6] Playwright Documentation, "Playwright - Fast and Reliable End-to-End Testing," <https://playwright.dev/>, 2024.
- [7] 경희대학교 컴퓨터공학부, "학과 공지사항", 컴퓨터공학과 홈페이지 게시판, <https://ce.khu.ac.kr/ce/user/bbs/MSR00040/list.do?menuNo=1600045>, 2024.